Manual del Programador

Modelado de `struct` y operador flecha (`->`) en C++

Autores

Federico Mattia
Máximo Orue
Ulises Ferrari
Lucas del Pino

1. Introducción

Este manual documenta el programa `EntidadProyecto`, escrito en C++. El programa define una estructura (`struct`) que agrupa tres datos de distinto tipo, pasa su dirección de memoria a una función y completa sus campos mediante el **operador flecha (`->`)**. Al final muestra los datos cargados y la dirección de memoria donde vive la estructura.

El manual está dirigido a programadores que necesiten entender, mantener o ampliar el código.

2. Objetivos del programa

- Modelar una entidad con un `struct` que combina `int`, `char[]` y `float`.
- Practicar el paso de una estructura **por puntero** a una función.
- Usar el operador `->` para acceder a los campos a través de un puntero.
- Verificar los datos ingresados y mostrar la dirección de memoria de la variable.

3. Requisitos

Elemento	Detalle
Lenguaje	C++ (estándar C++11 o superior)
Compilador	g++, clang++ o MSVC
Bibliotecas	`<iostream>` y `<cstring>`
Sistema operativo	Cualquiera con consola

Compilación con g++:

```
g++ main.cpp -o entidad
./entidad
```

4. Bibliotecas utilizadas

- `<iostream>`: entrada y salida por consola (`cout`, `cin`).
- `<cstring>`: funciones de cadenas de estilo C; en este programa se usa `strcpy`.
- `using namespace std;`: evita escribir el prefijo `std::` en cada instrucción.

5. Estructura de datos

```
struct EntidadProyecto {
    int id;
    char nombre[50];
    float metrica;
};
```

Campo	Tipo	Descripción
`id`	`int`	Identificador numérico de la entidad.
`nombre`	`char[50]`	Nombre o descripción. Admite hasta 49 caracteres más el terminador `'\0'`.
`metrica`	`float`	Valor decimal de operación.

Tamaño en memoria

En la mayoría de los compiladores de 32 o 64 bits, la estructura ocupa **60 bytes**: 4 del `int`, 50 del arreglo, 2 de relleno (padding) para alinear el `float` y 4 del `float`. El tamaño exacto depende del compilador y puede comprobarse con `sizeof(EntidadProyecto)`.

6. Prototipo de función

```
void cargarDatos(EntidadProyecto* ptr);
```

Se declara antes de `main()` para que el compilador conozca la función cuando se la invoca. Recibe un **puntero** a `EntidadProyecto`, por lo que modifica directamente la variable original y no una copia.

7. Función main()

7.1 Inicialización

```
EntidadProyecto miEntidad;
miEntidad.id = 0;
strcpy(miEntidad.nombre, "Vacio - Ulises Ferrari");
miEntidad.metrica = 0.0f;
```

Se crea la variable `miEntidad` y se le asignan valores iniciales. Como `nombre` es un arreglo, no se le puede asignar texto con `=`; por eso se usa `strcpy`. Con una variable (no puntero) se accede a los campos con el operador punto (`.`).

7.2 Encabezado

Se imprime un banner con el título del trabajo y el nombre del estudiante.

7.3 Carga de datos

```
cargarDatos(&miEntidad);
```

El operador `&` obtiene la dirección de memoria de `miEntidad` y la envía a la función.

7.4 Verificación

Tras la carga, el programa muestra `id`, `nombre` y `metrica`, y además `&miEntidad`, que imprime en hexadecimal la dirección de RAM donde se almacena la estructura.

8. Función cargarDatos()

```
void cargarDatos(EntidadProyecto* ptr) {
    cin >> ptr->id;
    cin.ignore();
    cin.getline(ptr->nombre, 50);
    cin >> ptr->metrica;
}
```

Instrucción	Función
`cin >> ptr->id;`	Lee un entero y lo guarda en el campo `id` de la estructura apuntada.
`cin.ignore();`	Descarta el salto de línea que `cin >>` deja en el búfer. Sin esta línea, `getline` leería una cadena vacía.

Instrucción	Función
`cin.getline(ptr->nombre, 50);`	Lee una línea completa, con espacios, hasta 49 caracteres.
`cin >> ptr->metrica;`	Lee un número decimal y lo guarda en `metrica`.

El operador flecha

`ptr->id` es equivalente a `(*ptr).id`: primero se desreferencia el puntero y luego se accede al campo. La flecha es la forma abreviada y más legible.

Situación	Operador	Ejemplo
Variable normal	`.`	`miEntidad.id`
Puntero a estructura	`->`	`ptr->id`

9. Flujo de ejecución

1. Se crea `miEntidad` y se inicializa con valores por defecto.
2. Se muestra el encabezado.
3. `main()` llama a `cargarDatos(&miEntidad)`.
4. La función pide ID, nombre y métrica, y los guarda con `->`.
5. Se vuelve a `main()`, que imprime los datos y la dirección de memoria.
6. El programa termina con `return 0;`.

10. Ejemplo de ejecución

```
======================================================
 MODELADO STRUCT - ESTUDIANTE: Federico Mattia
======================================================

-- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA --
=> Ingrese el ID de la entidad (entero): 7
=> Ingrese el Nombre o Descripcion: Sensor de temperatura
=> Ingrese la Metrica de Operacion (decimal/float): 23.5

=== DATOS VERIFICADOS EN LA MEMORIA RAM ===
ID Registrado: 7
Nombre Registrado: Sensor de temperatura
Metrica Guardada: 23.5
Direccion RAM Hexadecimal: 0x7ffd5c3a1b40
======================================================
```

La dirección hexadecimal cambia en cada ejecución.

11. Limitaciones conocidas

- **Sin validación de entrada:** si se escribe texto donde se espera un número, `cin` entra en estado de error y los campos quedan con valores inesperados.
- **Límite de nombre:** `getline` corta en 49 caracteres; el resto queda en el búfer.

- **Búfer pendiente:** después de leer `metrica` queda un salto de línea en el búfer; si se agregan más lecturas con `getline`, hay que volver a usar `cin.ignore()`.
- `strcpy` **sin control:** es seguro con el texto fijo actual (22 caracteres), pero con textos variables conviene usar `strncpy` o `std::string`.

12. Posibles mejoras

- Validar las entradas con `cin.fail()` y `cin.clear()` dentro de un bucle.
- Reemplazar `char nombre[50]` por `std::string`.
- Cargar un arreglo de entidades en lugar de una sola.
- Guardar y leer los registros desde un archivo con `fstream`.
- Mostrar `sizeof(EntidadProyecto)` para analizar el relleno de memoria.

13. Glosario

Término	Significado
`struct`	Tipo de dato definido por el programador que agrupa variables de distintos tipos.
Puntero	Variable que almacena la dirección de memoria de otra variable.
`&`	Operador de dirección: devuelve dónde está almacenada una variable.
`->`	Operador flecha: accede a un campo a través de un puntero a estructura.
Desreferenciar	Acceder al valor al que apunta un puntero mediante `*`.
Padding	Bytes de relleno que añade el compilador para alinear los campos.
Búfer	Zona de memoria temporal donde se guardan los datos de entrada.

Manual elaborado por Federico Mattia, Máximo Orue, Ulises Ferrari y Lucas del Pino.